

Árvores e Suas Generalizações

Aula 04 – Árvores binárias, árvores de busca



Árvores e suas Generalizações

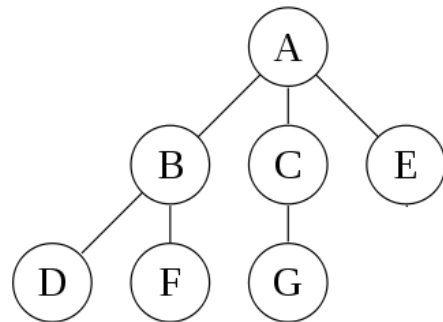
Árvores

As árvores são estruturas de dados hierárquicas amplamente utilizadas em computação. Elas são compostas por nós conectados por arestas, sendo um tipo de grafo acíclico direcionado. Nesta aula, abordaremos as **Árvores Binárias** e as **Árvores Binárias de Busca (BST - Binary Search Tree)**.

Em computação, uma árvore é um modelo abstrato de uma estrutura hierárquica, trata-se de uma estrutura não-linear constituída de nós com relações de parentesco (pai-filho).

Normalmente são utilizadas em:

- S.O. (arquivos);
- Linguagens (O.O.);
- etc.



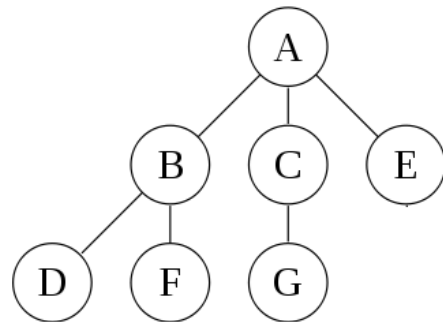
Árvores

Terminologias utilizadas:

- Árvore vazia: árvore que ainda não foi criada;
- Nós: cada item de uma árvore;
- Raiz: primeiro item da árvore;
- Folhas: Nós que não possuem filhos.

Regras:

- Você não pode conectar a raiz direto a folha, exceto se for uma árvore de nível = 1;
- Cada nó, é uma sub-árvore;
- No nosso exemplo: "A" é **pai** de "B", logo "B" é **filho** de "A";
- No nosso exemplo "E" é descendente de "A";
- No nosso exemplo "A" é ascestral de "E".



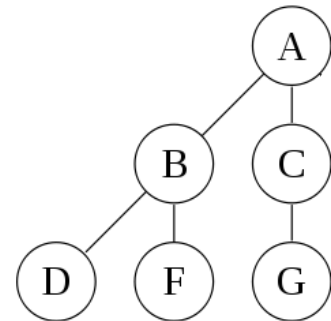
Árvores Binárias

Árvores Binárias

- Cada nó pode ter no máximo dois filhos (esquerda e direita);
- A profundidade de um nó é a distância até a raiz;
- A altura da árvore é a distância entre a folha mais distante e a raiz.

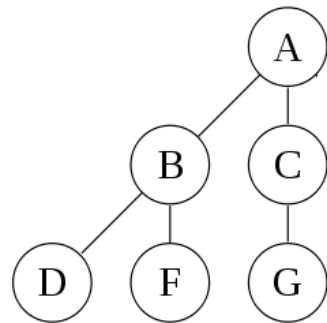
// Definição da estrutura de um nó da árvore binária

```
typedef struct No {  
    int valor;  
    struct No* esquerda;  
    struct No* direita;  
} No;
```



Árvores Binárias - Criando Nós

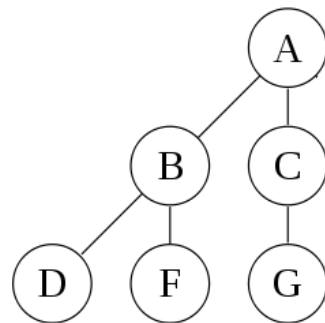
```
// Função para criar um novo nó  
No* criarNo(int valor){  
    No* novoNo = (No*)malloc(sizeof(No));  
    novoNo->valor = valor;  
    novoNo->esquerda = NULL;  
    novoNo->direita = NULL;  
    return novoNo;  
}
```



Percursos em Árvore

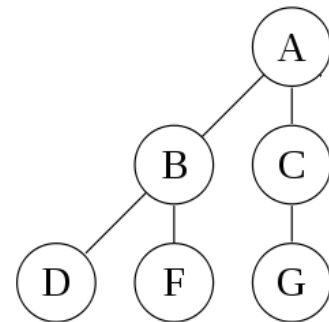
Percursos em Árvores

- Pré-ordem (raiz, esquerda, direita);
- Em-ordem (esquerda, raiz, direita) → usado para obter elementos ordenados em BST;
- Pós-ordem (esquerda, direita, raiz).



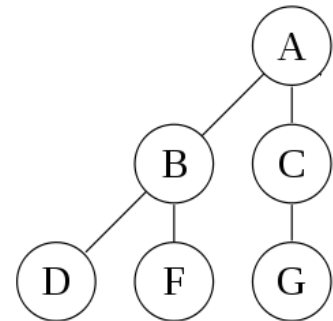
Pré-Ordem

```
void preOrdem(No* raiz) {  
    if (raiz != NULL) {  
        cout << raiz->valor;  
        preOrdem(raiz->esquerda);  
        preOrdem(raiz->direita);  
    }  
}
```



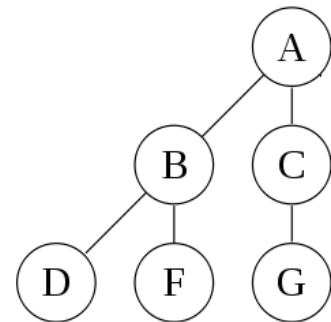
Em-Ordem

```
void emOrdem(No* raiz) {  
    if (raiz != NULL) {  
        emOrdem(raiz->esquerda);  
        cout << raiz->valor;  
        emOrdem(raiz->direita);  
    }  
}
```



Pós-Ordem

```
void posOrdem(No* raiz) {  
    if (raiz != NULL) {  
        posOrdem(raiz->esquerda);  
        posOrdem(raiz->direita);  
        cout << raiz->valor;  
    }  
}
```



Exercícios Arvores

Vamos criar uma arvore que tenha os seguintes elementos:

- Valor 15 é a raiz;
- Valor 10 é filho de 15 a esquerda;
- Valor 50 é filho de 15 a direita;
- Valor 20 é filho de 50 a esquerda.

Em seguida execute os algoritmos Pré-Ordem, Em-Ordem e Pós-Ordem e veja o resultado.



C . E . S . A . R

Pessoas impulsionando inovação.
Inovação impulsionando negócios.

